

Evaluation of Big Data Platforms and Data Warehouse Implementation

100752659

PROCESSING BIG DATA (7CS516) | WORD COUNT: 2725

Table of Contents

Abstract.....	2
Introduction.....	3
Overview of Data Warehouse.....	3
Cloud-Based Data Warehouse	3
Overview of the Big Data Cloud Platform Investigated.....	4
Platform Investigation and Evaluation Criteria	4
AWS Redshift	4
Google BigQuery	4
Investigation methodology.....	4
Platform Investigation.....	5
Amazon Web Services (AWS) - Amazon Redshift.....	5
Google Cloud Platform (GCP) – BigQuery	6
Platform Comparative Analysis.....	6
Comparison Table	6
Business Recommendations.....	7
Recommendation and Conclusion	8
Big Data Processing and Analytics.....	8
Introduction.....	8
Platform Used	8
Data Preparation: Cleaning, Standardization, and Integrity Checks.....	8
Data quality checks and data cleaning process	10
Business and Research Questions/Insights	11
Machine Learning (Classification) - Model development	15
Research Question	15
Model Selection: Random Forest Classifier	15
Features Selection	15
Feature Engineering	16
Model Performance Evaluation	16
Confusion matrix	17
Hyperparameter Tuning (Model Optimization).....	17
Summary Conclusion.....	18
Recommendations.....	18
Future Work.....	18
Conclusion	18
References.....	19

Abstract

In this report, I'll be evaluating Amazon Web Services Redshift (AWS) and Google BigQuery, two leading cloud data warehouse platforms that currently have significant market shares in the data industry. The study evaluates these platforms based on the following key criteria: performance at scale, security, cost efficiency, ease of use, elasticity, and integration (Firebolt, 2024). Additionally, this report covers the processing of big data using PySpark to perform data pre-processing, data exploration, analysis, and also make predictive business outcomes. This evaluation and practical implementation offer an analysis of cloud data warehouse capabilities and their application to get actionable insights from historical data.

Keywords: Big Data, Cloud Data Warehousing, Amazon Redshift, Google BigQuery, Performance at Scale, Elasticity, Cost Efficiency, Data, PySpark, Exploratory Data Analysis, Machine Learning

Introduction

As global businesses increasingly rely on data for decision-making, the need to store, process and analyse huge amounts of data has surged exponentially. Cloud data warehouses such as Amazon Web Services Redshift (AWS) and Google BigQuery have emerged as critical tools for solving complex data analytics in big data. AWS Redshift and Google BigQuery hold a significant market share of 15.74% and 13.80%, respectively, and are recognised as leading solutions, making them ideal for this evaluation. These cloud data warehouses have become very important in enabling scalable analytics, according to a report on Cloud Database Management Systems (Gartner, 2024).

Overview of Data Warehouse

A data warehouse is a storage system designed to store large of structured and semi-structured data which is usually in a centralised repository within an organisation. These warehouses support analytics, reporting, data mining and storage and the warehouses can be implemented primarily in two ways on-premise or over the cloud (Google Cloud, 2024). Figure 1, shows an overview of the implementation of data warehouse architecture.

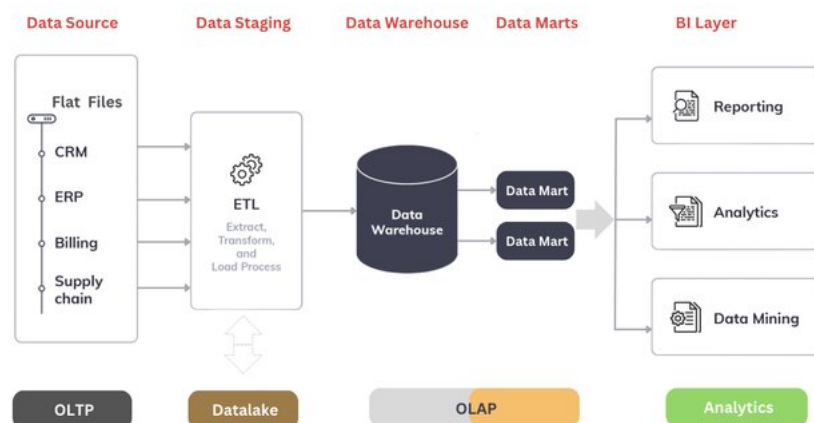


Figure 1 Data Warehouse Architecture (Amit Jaiswar)

Cloud-Based Data Warehouse

A cloud data warehouse is a centralised database in a public cloud for storing, processing, integrating, and managing large volumes of structured and semi-structured data optimised for scalable Business intelligence (BI) and advanced analytics. (Google Cloud, 2024). It can also help unify a warehouse and data lake to maintain enterprise data.

Overview of the Big Data Cloud Platform Investigated

Amazon Web Services (AWS) is a cloud computing platform offering services such as computing, storage, databases, machine learning, artificial intelligence tools, data lakes, and the Internet of Things (IoT). (AWS, 2024)

Google Cloud Platform (GCP) is a public cloud vendor that offers a collection offering computing services ranging from computing, storage and database, networking, and big data to machine learning and Virtual machines. (Pluralsight, 2024)

Platform Investigation and Evaluation Criteria

This section overviews AWS Redshift and Google BigQuery, two notable cloud-based data warehouse solutions designed to support the management and analysis of large-scale datasets.

AWS Redshift

Allows businesses to store, query, and analyse enormous amounts of structured and unstructured data (Cloudzero, 2024). With the advent of cloud computing and the increase in demand, Redshift has become a tool for businesses to use in their store, processing and analysing data.

Google BigQuery

BigQuery is a serverless data computing platform that offers services that help manage for managing and analysing extensive amounts of structured and unstructured data, it provides on-demand tools such as machine learning, artificial intelligence, and business intelligence. By handling infrastructure complexities, BigQuery allows businesses to focus on data analysis and interpretation. (Google Cloud, 2024)

Investigation methodology

This research investigates Redshifts and BigQuery by using a systematic approach, drawing from reputable sources. These include peer-reviewed journals, academic articles, official documentation from official AWS Web and Google Cloud Platform documentation, and online resources. The evaluation assesses both platforms based on: Performance at scale, Elasticity, Ease of use, Security, Cost efficiency, and Integration.

Platform Investigation

Amazon Web Services (AWS) - Amazon Redshift

Performance at Scale: Uses Massively Parallel Processing (MPP) to scale performance by distributing queries across multiple nodes, making it effective for structured and OLAP-style tasks using Massively Parallel Processing (MPP). It scales independently by decoupling storage and computing with RA3 instances. However, users must resize clusters to avoid downtime, and manual configurations are required for optimisation but AWS provides a serverless option for concurrency scaling.

Elasticity: Redshift is elastic but requires manual scaling, it automatically provides additional instantly at peak query loads to leverage concurrent scaling. Elasticity depends on manual cluster configuration, and scaling takes time. Standard Redshift lacks pause-and-resume features for cost savings. However, integration is not seamless, requiring data and workload migration to new environments (AWS, 2024).

Ease of Use: Based on PostgreSQL, which allows businesses familiar with SQL to interact easily with the web-based SQL editor, Redshift can integrate with other AWS services. While SQL-compatible, new businesses must learn AWS-specific features and optimisations. Advanced tasks require CLI/API usage (AWS, 2024). Redshift requires an administrator for cluster management, scaling, and optimisation.

Security: Redshift supports AES-256 encryption at rest and TLS encryption in transit, also, users control encryption keys for compliance and security needs. Redshift implements access control and authentication for identity access and enforces multi-factor authentication (MFA) for enhanced security. Redshift supports Amazon Virtual Private Cloud (VPC) for private network isolation but requires manual security configurations (AWS, 2024). While highly secure, businesses need security expertise for proper setup and maintenance.

Cost Efficiency: Redshift operates on an on-demand pricing model, charging businesses based on compute node hours based on the cluster size, businesses can also get to pay upfront for one to three years reserved, saving up to 75%, Redshifts provides free concurrency scaling credits, charging only when usage exceeds free limits. Idle clusters continue accruing costs unless manually resized. AWS Redshift also offers region-specific pricing, varying by location (e.g., Africa vs. Europe).

Google Cloud Platform (GCP) – BigQuery

Performance at Scale: Automatically manages resources without infrastructure provisioning, making it easy to handle huge workloads, It delivers fast query performance on exabyte-scale data using Dremel columnar storage and tree-based query execution. Unlike Redshift, it may introduce latency in complex joins and aggregations.

Elasticity: BigQuery is fully serverless and auto-scalable, automatically allocating and deallocating resources based on query needs without downtime or human intervention. Queries are distributed dynamically across infrastructure for better scaling.

Ease of Use: BigQuery has a simple, user-friendly interface with drag-and-drop analysis capabilities with Google Data Studio. It uses Standard SQL, making it accessible for SQL-based businesses. However, it offers less control over indexing and query execution compared to Redshift. The learning curve is minimal for those familiar with the Google Cloud ecosystem.

Security: BigQuery encrypts data by default at rest and in transit using AES-256, also provides automated security threat detection and doesn't require a security expert like Redshift, but that also makes it rigid and not flexible. BigQuery meets the following compliance standards: PCI DSS, GDPR, SOC 1/2/3, and ISO 27001.

Cost Efficiency: Offers a pay-per-query pricing model by only charging for queries executed making it cost-effective for businesses, and they also offer monthly flat-rate pricing for high-volume queries. There's no pay for idle resources unlike Redshift, however, it also might not be suitable for businesses that require frequently accessed data.

Platform Comparative Analysis

Comparison Table

Table 1 Comparative Analysis of AWS Redshift and Google BigQuery

Criteria	AWS Redshift	GCP BigQuery
Performance at Scale	Optimized for structured workloads with MPP. Requires manual scaling.	Fully serverless with real-time query execution

Elasticity	Manual scaling is required; auto-scaling is available with limitations.	Fully serverless and auto-scalable.
Ease of Use	SQL-compatible, requires AWS-specific knowledge.	User-friendly UI with drag-and-drop features.
Cost Efficiency	Reserved instances save costs; idle resources still incur charges.	Pay-per-query model is cost-effective for infrequent workloads.
Security	Strong encryption, requires manual configurations.	Automated encryption, meets multiple compliance standards.
Integration	Seamless with AWS services, and limited multi-cloud support.	Supports multi-cloud integration and BI tools.

Business Recommendations

Table 2 Business Recommendation Table

Business Size	Recommended Platform	Justification
Small	GCP BigQuery	It is cost-effective for low-frequency queries, serverless model eliminates infrastructure management, easy BI integration making a good fit for small Business.
Medium	AWS Redshift	Has balanced performance and scalability, reserved instances provide cost efficiency for predictable workloads. it's flexibility makes it suitable for a medium-sized business
Large	AWS Redshift	Best for high-performance, large-scale workloads, deep security controls, compliance, and enterprise support. It's flexibility also makes it a great fit for large businesses.

Recommendation and Conclusion

AWS Redshift and Google BigQuery are powerful cloud data warehouses with distinct strengths. Redshift is ideal for structured, long-running analytical workloads with fine-grained control, while BigQuery excels in real-time, ad hoc querying with seamless auto-scaling. For businesses, the choice depends on workload patterns, costs, integration and security needs. For businesses seeking flexibility, BigQuery is preferable unless operating at a large scale.

Big Data Processing and Analytics

Introduction

This section covers an in-depth analysis and exploration of the data provided, which includes five facts and 16 dimension tables. The objective is to analyse product and sales performance and predict product category orders based on sales conditions.

Platform Used

For this project, I chose PySpark because it provides speed, ease of use, and SQL compatibility. It also manages complex operations by handling large datasets efficiently. PyCharm was chosen for this project due to its intelligent code assistance, providing seamless integration with GitHub and Git for Version control, it also has built-in support for PySpark.

Data Preparation: Cleaning, Standardization, and Integrity Checks

Displaying the Schema to have an overview of the structure of the datasets.

```
Schema for DimProduct:
root
|-- ProductKey: integer (nullable = true)
|-- ProductAlternateKey: string (nullable = true)
|-- ProductSubcategoryKey: integer (nullable = true)
|-- WeightUnitMeasureCode: string (nullable = true)
|-- SizeUnitMeasureCode: string (nullable = true)
|-- EnglishProductName: string (nullable = true)
|-- SpanishProductName: string (nullable = true)
|-- FrenchProductName: string (nullable = true)
|-- StandardCost: double (nullable = true)
|-- FinishedGoodsFlag: boolean (nullable = true)
```

Figure 2 Displaying the Schema of All Datasets in PySpark

Standardising Column Names to Snake Case to avoid having special characters issues when performing joining operations on the datasets. It also makes it more readable and clearer and helps avoid issues that may arise from using frameworks like PySpark and Pandas.

```

Updated columns in DimProduct: ['product_key', 'product_alternate_key', 'product_subcategory_key',
'weight_unit_measure_code', 'size_unit_measure_code', 'english_product_name', 'spanish_product_name',
'french_product_name', 'standard_cost', 'finished_goods_flag', 'color', 'safety_stock_level', 'reorder_point',
'list_price', 'size', 'size_range', 'weight', 'days_to_manufacture', 'product_line', 'dealer_price', 'class',
'style', 'model_name', 'english_description', 'start_date', 'end_date', 'status']
Updated columns in FactInternetSales: ['product_key', 'order_date_key', 'due_date_key', 'ship_date_key',
'customer_key', 'promotion_key', 'currency_key', 'sales_territory_key', 'sales_order_number',
'sales_order_line_number', 'revision_number', 'order_quantity', 'unit_price', 'extended_amount',
'unit_price_discount_pct', 'discount_amount', 'product_standard_cost', 'total_product_cost', 'sales_amount',
'tax_amt', 'freight', 'carrier_tracking_number', 'customer_p_o_number', 'order_date', 'due_date', 'ship_date']
Updated columns in DimCustomer: ['customer_key', 'geography_key', 'customer_alternate_key', 'title',
'first_name', 'middle_name', 'last_name', 'name_style', 'birth_date', 'marital_status', 'suffix', 'gender',

```

Figure 3 Standardizing Column Names to Snake Case in PySpark

In this step, we verify that the key columns in all the different datasets have consistent datatypes.

```

Checking data types in FactInternetSales:
root
|-- product_key: integer (nullable = true)
|-- customer_key: integer (nullable = true)
|-- order_date_key: integer (nullable = true)
|-- sales_territory_key: integer (nullable = true)

Checking data types in DimProduct:
root
|-- product_key: integer (nullable = true)
|-- product_subcategory_key: integer (nullable = true)

```

Figure 4 Results Verification of Columns datatype

Here we assess all the columns to make sure we don't have null values that could affect any join or aggregation operations.

```

Checking for NULL values in FactInternetSales:

product_key has 0 NULL values in FactInternetSales
customer_key has 0 NULL values in FactInternetSales
order_date_key has 0 NULL values in FactInternetSales
sales_territory_key has 0 NULL values in FactInternetSales

=====

Checking for NULL values in DimProduct:
product_key has 0 NULL values in DimProduct
product_subcategory_key has 0 NULL values in DimProduct

=====

```

Figure 9 Results of Null Value Check-in Key Columns

After verifying and validating our data, we then proceeded to join key dimension tables without fact tables (FactInternetSales) to enhance our sales and product analysis. Here is the list of dimension tables we're making use of for sales performance: DimProduct, DimCustomer, DimDate, DimSalesTerritory, DimProductSubcategory, and DimProductCategory.

Table 3 Joined Dataset for Sales and Product Analysis

sales_territory_key	customer_key	product_key	order_date_key	due_date_key	ship_date_key	promotion_key	currency_key
6	21768	310	20101229	20110110	20110105	1	
7	28389	346	20101229	20110110	20110105	1	
1	25863	346	20101229	20110110	20110105	1	
4	14501	336	20101229	20110110	20110105	1	
9	11003	346	20101229	20110110	20110105	1	
4	27645	311	20101230	20110111	20110106	1	
9	16624	310	20101230	20110111	20110106	1	
9	11005	351	20101230	20110111	20110106	1	
9	11011	344	20101230	20110111	20110106	1	
4	27621	312	20101231	20110112	20110107	1	

Data quality checks and data cleaning process

Ensuring Data Integrity: Row Count Validation Before and After Joining, here the results shows that the FactInternetSales and the joined tables has the same number of rows (60398) as seen on the Figure 9.

```

1 # Check row count before and after joining
2 fact_sales_count = datasets["FactInternetSales"].count()
3 merged_fact_sales_count = fact_sales.count()
4
5 print(f"FactInternetSales before join: {fact_sales_count} rows")
6 print(f"Merged dataset after join: {merged_fact_sales_count} rows")
7
✓ [81] 1s 181ms
FactInternetSales before join: 60398 rows
Merged dataset after join: 60398 rows

```

Figure 10 Validating Row Consistency Post-Join

Rechecking the merged tables for duplicate rows, the result returns a 0 which means we don't have any duplicated rows.

```

1 # Check for duplicate rows
2 duplicate_count = fact_sales.groupby(fact_sales.columns).count().filter("count > 1").count()
3 print(f"Duplicate rows in merged dataset: {duplicate_count}")
✓ [109] 17s 611ms
[Stage 892:> (0 + 4) / 4]
Duplicate rows in merged dataset: 0

```

Figure 11 Duplicate Row Check in Merged Dataset

Next, we check for null values on columns that are important to us for our analysis.

```

2 # List of important columns to check
3 important_columns = ["product_key", "customer_key", "order_date_key", "sales_territory_key", "sales_amount"]
4
5 # Check for NULL values in important columns
6 for col_name in important_columns:
7     null_count = fact_sales.filter(col(col_name).isNull()).count()
8     print(f"{col_name} has {null_count} NULL values")
✓ [83] 4s 299ms

product_key has 0 NULL valuesproduct_key has 0 NULL values
customer_key has 0 NULL valuesproduct_key has 0 NULL values
customer_key has 0 NULL values
order_date_key has 0 NULL valuesproduct_key has 0 NULL values
customer_key has 0 NULL values
order_date_key has 0 NULL values
sales_territory_key has 0 NULL valuesproduct_key has 0 NULL values

```

Figure 12 Null Value Check for Key Sales Columns

Table 4 Descriptive Statistics of Key Sales Metrics

```
fact_sales.select("order_quantity", "unit_price", "sales_amount", "tax_amt", "freight").describe().show()
```

✓ [84] 1s 476ms

summary	order_quantity	unit_price	sales_amount	tax_amt	freight
count	60398	60398	60398	60398	60398
mean	1.0	486.0869105053777	486.0869105053777	38.88695370873952	12.152217111493709
stddev	0.0	928.489891980807	928.489891980807	74.27919255248155	23.212248232767838
min	1	2.29	2.29	0.1832	0.0573
max	1	3578.27	3578.27	286.2616	89.4568

Business and Research Questions/Insights

For the sales and product analysis, we'll be focusing on the following business questions;

1. Which products generate the highest sales revenue and profit margins?

Table 5 Top-Selling Products by Revenue and Profit Margin

english_product_name	total_sales	profit_margin
Mountain-200 Blac...	1373469.5481999982	626621.5735000019
Mountain-200 Blac...	1363142.093399998	621759.6083000018
Mountain-200 Silv...	1339462.790399998	610864.4347999995
Mountain-200 Silv...	1301100.0983999984	593490.4727999989
Mountain-200 Blac...	1294866.141199998	590477.4604000015
Mountain-200 Silv...	1257434.572799998	573511.9375999994
Road-150 Red, 48	1205876.9900000044	474150.84460000356
Road-150 Red, 62	1202298.7200000044	472743.86880000355
Road-150 Red, 52	1080637.5400000038	424906.69160000305
Road-150 Red, 56	1055589.6500000036	415057.86100000294

Table 5 and Figure 11 shows the total sales and profit margins at a category level. The bikes category has the highest total sales and profit margins, followed by accessories and clothing.

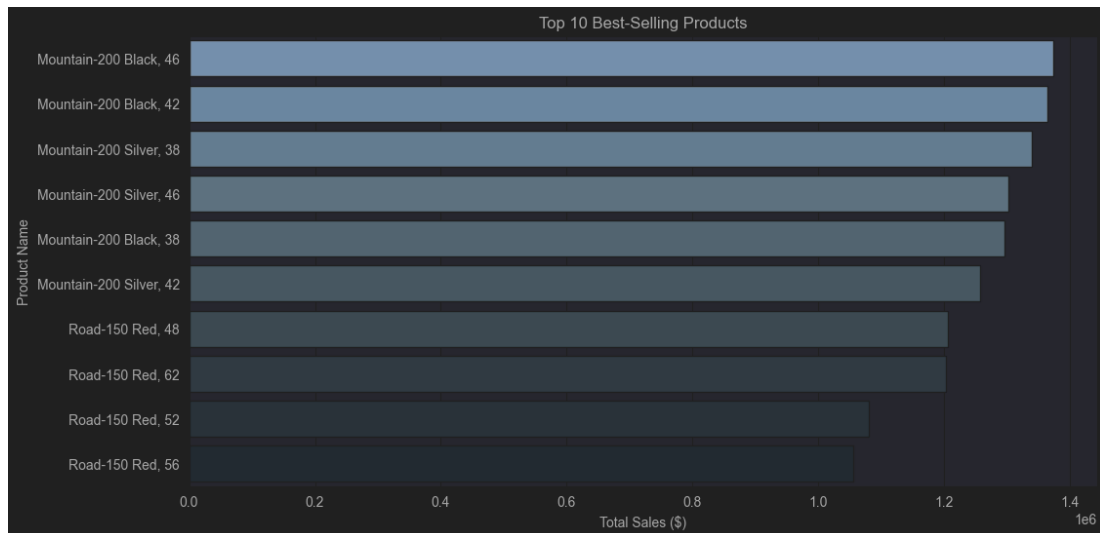


Figure 13 Top-Selling Products

Insight: Bikes generate the highest sales and profit margin, outperforming other product categories. Within the subcategories, Road Bikes and Mountain Bikes are the top contributors to sales. Accessories and Clothing have lower sales and profit margins.

2. How does product performance vary across different categories and subcategories?

Table 6 Sales and Profit by Product Category

english_product_category_name	total_sales	profit_margin
Bikes	2.8318144650701128E7	1.1505796500600025E7
Accessories	700759.9599999422	438674.56809997605
Clothing	339772.6100000149	136412.57629999678

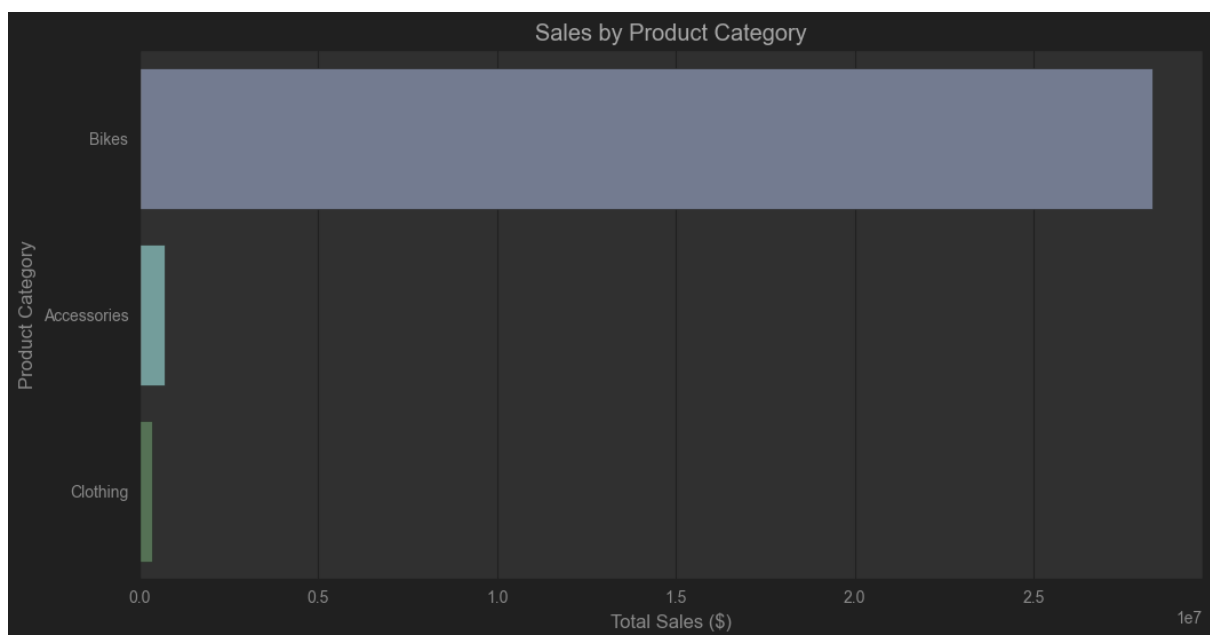


Figure 14 Profit Margins by Product Category

Table 7 Sales Performance by Product Subcategory

10 rows ▾ 10 rows × 3 cols			Static Output
english_product_category_name	english_product_subcategory_name	total_sales	
Bikes	Road Bikes	1.4520584036299847E7	
Bikes	Mountain Bikes	9952759.564400222	
Bikes	Touring Bikes	3844801.0500000035	
Accessories	Tires and Tubes	245529.32000000443	
Accessories	Helmets	225335.60000000076	
Clothing	Jerseys	172950.67999999886	
Clothing	Shorts	71319.81000000038	
Accessories	Bottles and Cages	56798.19000000005	
Accessories	Fenders	46619.57999999933	
Accessories	Hydration Packs	40307.66999999987	

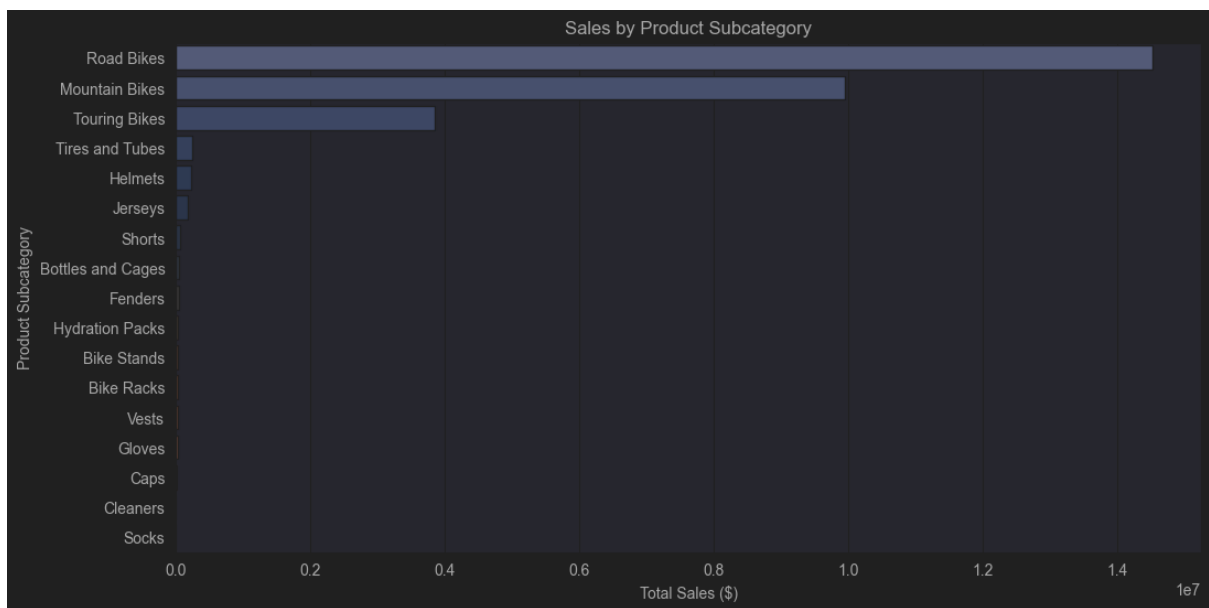


Figure 5 Plot Sales by Subcategory

Insight: The top-selling products are Mountain-200 Black and Mountain-200 Silver, contributing the highest sales revenue of 27.01%. These products also show strong profit margins, indicating their profitability.

3. How do sales trends vary across different sales territories?

Table 8 Sales Distribution Across Territories

3 rows ▾ 3 rows × 2 cols			Static Output
sales_territory_group	total_sales		
North America	1.1367634372902095E7		
Pacific	9061000.584400563		
Europe	8930042.263400966		

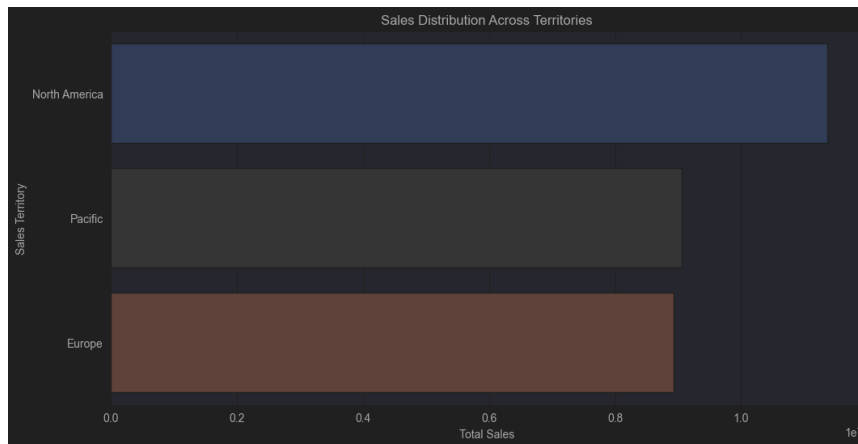


Figure 6 Sales Distribution Across Territories

Insight: The analysis shows that North America has the highest total sales, followed by the Pacific and Europe. This indicates that sales performance is strongest in North America, suggesting a larger market presence or demand in this region. Marketing efforts and inventory distribution can be strategized.

4. How does customer income level impact total sales?



Figure 7 Sales Distribution Across Customer Income Levels

Insight: The middle-income customers earning between \$40,000 and \$70,000 generate the highest sales, while both lower and higher-income groups contribute significantly less. This suggests that the sales optimisation should target the mid-range income earners.

5. Which occupations contribute the most to total sales?

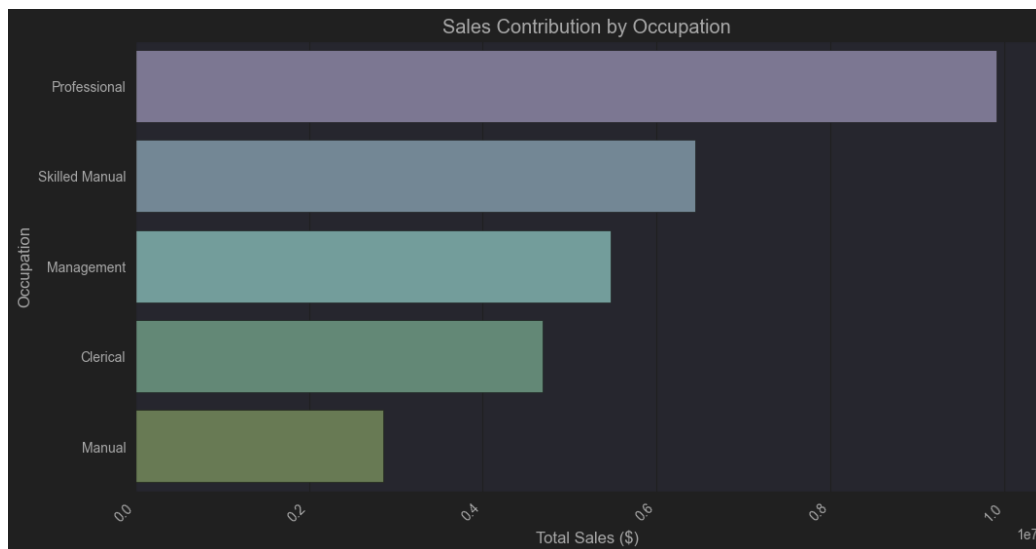


Figure 8 Sales Contribution by Occupation

Insight: The highest sales revenue is contributed by professionals, followed by skilled manual and management occupations. This suggests that targeting professionals with premium products or specialised marketing campaigns could be beneficial for the revenue growth of the business.

Machine Learning (Classification) - Model development

Research Question

Can we predict the product category based on sales-related features?

Model Selection: Random Forest Classifier

I chose the Random Forest classifier because it is capable of capturing complex, non-linear relationships between features and target variables. Due to its ensemble nature, it performs well without extensive hyperparameter tuning, reduces overfitting and increases generalisation by combining multiple decision trees to improve overall performance.

Features Selection

Relevant features were selected, including sales_territory_group, order_quantity, sales_amount, and freight, while the target variable was english_product_category_name to set the foundation for training the classification model.


```
# Convert category name to numerical labels
indexer = StringIndexer(inputCol="english_product_category_name", outputCol="label")
labeled_data = indexer.fit(labeled_data).transform(labeled_data)
labeled_data.show()
```

✓ [94] 4s 682ms

customer_key	sales_territory_group	order_quantity	sales_amount	freight	english_product_category_
25886	North America	1	24.49	0.6123	Clothing
19225	North America	1	63.5	1.5875	Clothing
23105	Pacific	1	24.49	0.6123	Clothing
16944	Europe	1	24.49	0.6123	Clothing
29356	Europe	1	63.5	1.5875	Clothing
28207	Pacific	1	63.5	1.5875	Clothing
22432	Europe	1	24.49	0.6123	Clothing
21349	North America	1	24.49	0.6123	Clothing
21017	Europe	1	24.49	0.6123	Clothing
11088	North America	1	24.49	0.6123	Clothing
11041	North America	1	63.5	1.5875	Clothing

Figure 9 Encoding Product Categories

Feature Engineering

In this step, "VectorAssembler" was used to combine relevant numerical features into a single vector column, the transformation is essential as models require input features to be vectorised, making it effective for the model to learn.

Model Performance Evaluation

The Random Forest classifier outperformed the other models, achieving the highest accuracy of 94.71%, followed by Logistic Regression (85.72%) and Naïve Bayes (83.99%), the model was trained and evaluated on the dataset to predict product categories based on sales-related features. This indicates that Random Forest is the most effective model for this task.

```
Random Forest Accuracy: 0.947139813145386
Logistic Regression Accuracy: 0.8571545648254385
Naïve Bayes Accuracy: 0.839944271430913
```

Figure 10 Model Accuracy Comparison

Further evaluation shows that the Random Forest classifier demonstrated strong overall performance with an F1-Score of 94.73%, Precision of 94.76%, and Recall of 94.71%. These results indicate that the model is both precise and effective at correctly predicting product categories, making it the most suitable choice for this task.

```
F1-Score: 0.9473441362754716
Precision: 0.9475840282200609
Recall: 0.947139813145386
```

Figure 11 Performance Metrics of the Random Forest Model

Confusion matrix

The confusion matrix shows that the Random Forest Model performs well, with minimal misclassification errors.

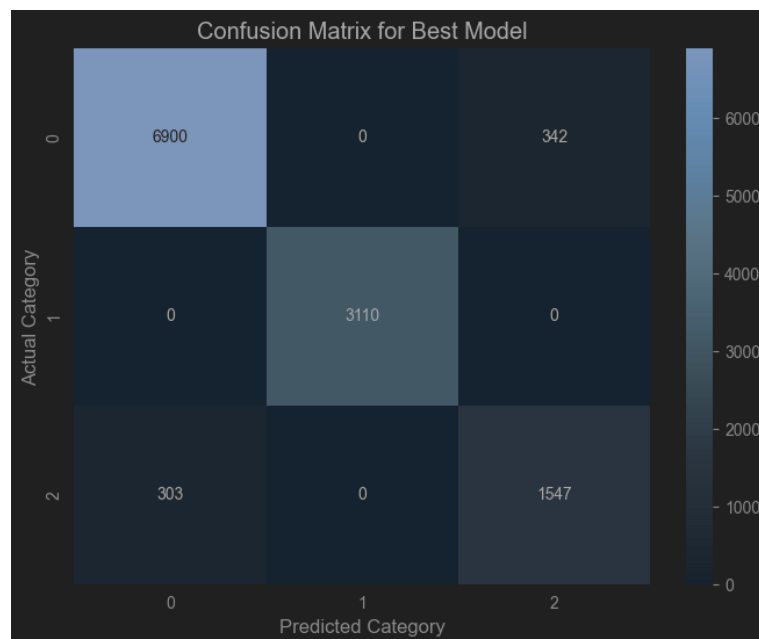


Figure 12 Confusion Matrix for Product Category Prediction

Hyperparameter Tuning (Model Optimization)

The outcome suggests that the model has an optimal balance of bias and variance, meaning that the chosen default hyperparameters were well-suited for the dataset, also tuning typically improves models that suffer from overfitting or underfitting.

```
Tuned Random Forest F1-Score: 0.9473441362754716  
Tuned Random Forest Precision: 0.9475840282200609  
Tuned Random Forest Recall: 0.947139813145386
```

Figure 13 Tuned Random Forest Model Performance

Summary Conclusion

This analytics section showed a comprehensive data pre-processing, exploration, and machine learning analysis to extract insights and build predictive models for sales performance and product categorisation. The insights obtained from this analysis can assist businesses in making data-driven decisions regarding sales strategies, inventory management, and marketing efforts. The machine learning model provides a reliable framework for predicting product categories, which can be further refined with additional features or datasets for enhanced business intelligence.

Recommendations

Based on the findings from our data analysis and machine learning model, I propose that the business optimise and prioritise the high-demand products, insights from top-selling categories and regional performance can be leveraged on, also the logistics and freight costs can be minimised by analysing shipping distribution channels

Future Work

To improve accuracy further advanced machine-learning models such as LSTM, and RNNs can be implemented to capture complex patterns in sales performance. Also, to explore connecting PySpark with advanced cloud storage platforms such as Google Cloud Storage to enhance scalability and data processing efficiency.

Conclusion

This project evaluated BigQuery and Redshift for cloud-based data warehousing, assessing their scalability, performance, and cost-efficiency for big data processing. Additionally, PySpark was utilized for data analytics, transformation and machine learning, providing key insights into sales trends, product performance, and regional sales, enabling data-driven decision-making for business optimization.

References

6sense (2024) *Market Share for Data Warehousing Technologies*. Available at: <https://6sense.com/tech/data-warehousing> [Accessed 20 Dec. 2024].

Borra, P., 2024. *Comprehensive Survey of Amazon Web Services (AWS): Techniques, Tools, and Best Practices for Cloud Solutions*. ResearchGate. Available at: https://www.researchgate.net/profile/Praveen-Borra/publication/381956331_Comprehensive_Survey_of_Amazon_Web_Services_AWS_Techniques_Tools_and_Best_Practices_for_Cloud_Solutions/ [Accessed 3 Jan. 2025].

Amazon Web Services (2023) *Amazon Redshift: Security and Server-Side Encryption*. Available at: <https://docs.aws.amazon.com/redshift/latest/mgmt/security-server-side-encryption.html> [Accessed 2024].

Amazon Web Services (2023) *Amazon Redshift Data API*. Available at: <https://docs.aws.amazon.com/redshift/latest/mgmt/data-api.html> [Accessed 2024].

Amazon Web Services (2023) *Big Data Analytics Options on AWS - Amazon Redshift*. Available at: <https://docs.aws.amazon.com/whitepapers/latest/big-data-analytics-options/amazon-redshift.html> [Accessed 2024].

Amazon Web Services (2023) *Welcome to Amazon Redshift*. Available at: <https://docs.aws.amazon.com/redshift/latest/mgmt/welcome.html> [Accessed 2024].

CloudZero (2024) *AWS Redshift Guide: Use Cases, Pros and Cons, and Pricing*. Available at: <https://www.cloudzero.com/blog/aws-redshift/> [Accessed 13 Dec. 2024].

Firebolt (2024) *BigQuery vs Redshift: A Comprehensive Comparison of Firebolt AWS Features*. Available at: <https://www.firebolt.io/comparison/firebolt-vs-redshift> [Accessed 10 July 2024].

Firebolt (2024) *Cloud Data Warehouse Evaluation Guide*. Available at: <https://www.firebolt.io/resources/cloud-data-warehouse-evaluation-guide> [Accessed 13 Dec. 2024].

Google Cloud (2023) *What is a Data Warehouse?* Available at: <https://cloud.google.com/learn/what-is-a-data-warehouse> [Accessed 2024].

Google Cloud (2023) *BigQuery Row-Level Security*. Available at: <https://cloud.google.com/bigquery/docs/row-level-security-intro> [Accessed 2024].

Quantiphi (2024) *Modern Data Strategy: Analyzing Amazon Redshift Across Sectors*. Available at: <https://quantiphi.com/modern-data-strategy-analyzing-amazon-redshift-across-sectors/> [Accessed 13 Dec. 2024].

Astera (2023) *An Overview of Cloud Data Warehouses: Amazon Redshift (AWS), Google BigQuery (GCP), and Microsoft Azure Synapse*. Available at:
<https://www.astera.com/type/blog/bigquery-vs-redshift/> [Accessed 2024].

Integrate.io (2023) *Redshift vs BigQuery: A Comprehensive Guide*. Available at:
<https://www.integrate.io/blog/redshift-vs-bigquery-comprehensive-guide/> [Accessed 2024].

GeeksforGeeks (2023) *AWS Redshift vs Google BigQuery*. Available at:
<https://www.geeksforgeeks.org/aws-redshift-vs-google-bigquery/> [Accessed 2024].

mParticle (2023) *BigQuery vs Redshift*. Available at:
<https://www.mparticle.com/blog/bigquery-vs-redshift/> [Accessed 2024].

Stitch Data (2023) *Redshift vs BigQuery*. Available at:
<https://www.stitchdata.com/resources/redshift-vs-bigquery/> [Accessed 2024].

ProjectPro (2023) *Redshift vs BigQuery*. Available at:
<https://www.projectpro.io/article/redshift-vs-bigquery/680> [Accessed 2024].